

Chronologisch examen-stappenplan

Dag 1 – Plannen

Stap 1: Casus doorlezen

Lees de casus rustig door en markeer de belangrijkste informatie.

Let vooral op:

- wat de applicatie moet kunnen;
- wie de gebruikers zijn (rollen);
- welke gegevens opgeslagen moeten worden;
- mogelijke entiteiten en attributen;
- mogelijke schermen / functionaliteiten.

Stap 2: Entiteiten en attributen zoeken

Maak alvast op papier een simpele voorbeeldtabel met entiteiten en attributen. Dit is nog niet definitief, maar helpt straks bij NORMA en SQL.

Stap 3: D.O.D invullen

Vul de Definition of Done in volgens het template.

Zet hierin minimaal:

- inleiding / korte uitleg van de opdracht;
- betrokkenen;
- doelstelling;
- benodigdheden;
- te realiseren functionaliteiten (usecases);
- gebruikte technieken;
- code conventions.

(als je er niet uit komt, je hebt een voorbeeld D.O.D bij je bestanden)

Stap 4: Usecases maken in Visio

Maak de hoofdprocessen en deelprocessen. Gebruik ook Excel voor de rollen wie wel iets kan en niet iets kan.

Stap 5: User stories maken in Word

Maak user stories vanuit de casus.

Gebruik simpele vorm:

Als [gebruiker] wil ik [actie] zodat [doel].

Voorbeeld: Als medewerker wil ik een klant kunnen toevoegen, zodat nieuwe klanten opgeslagen kunnen worden.

Stap 6: Planning maken voor het examen

Maak een realistische word document planning voor de 7 examendagen. Hou deze planning aan uit dit bestand.

Dag 2 – Ontwerpen

Stap 1: User stories in DevOps maken

Zet de user stories van dag 1 in DevOps. Dit gebruik je later ook om je voortgang bij te houden.

Stap 2: NORMA maken / ORM / ERD

Maak het echte database-ontwerp in NORMA. Gebruik je voorbeeldtabellen van dag 1 als basis.

Gebruik het NORMA stappenplan!

Stap 3: Wireframes maken

Maak simpele schetsen van de schermen die je nodig hebt.

Bijvoorbeeld:

- hoofdmenu;
- overzichtsscherm;
- toevoegen-scherm;
- wijzigen-scherm;
- eventueel login-scherm;

Het hoeft niet mooi te zijn. Het moet duidelijk maken welke knoppen, velden, comboboxen en lijsten op het scherm komen. Gebruik je use cases en user stories als basis.

Stap 4: Activiteitendiagrammen maken

Maak in totaal 3 activiteitendiagrammen voor de moeilijkere functionaliteiten.

Maak vooral diagrammen voor processen met meerdere stappen of keuzes, zoals:

- toevoegen
- update
- inloggen eventueel.

Niet voor elke kleine knop overdrijven. Vooral maken voor processen met meerdere stappen of keuzes.

Stap 5: Ontwerpkeuzes kort onderbouwen

Maak kort duidelijk word document waarom je ontwerp logisch is.

- Benoem minimaal:
- waarom deze tabellen nodig zijn;
- waarom deze relaties nodig zijn;
- waarom deze schermen nodig zijn;
- welke persoonsgegevens worden opgeslagen;
- hoe je rekening houdt met privacy;
- hoe je veilig werkt, bijvoorbeeld met parameterized queries en login/rollen;
- waarom dit haalbaar is binnen de examentijd.

Dit is belangrijk, omdat de beoordeling vraagt dat je ontwerpkeuzes onderbouwd zijn met haalbaarheid, privacy en security.

Dag 3 – Realiseren

Stap 1: DevOps koppelen aan Visual Studio

Clone het project vanuit DevOps naar Visual Studio (bekijk aantekeningen schrift hiervoor)

Maak eventueel je eigen branch aan, zodat je niet direct op main werkt.

- Werk volgens de board flow;
- taak oppakken;
- user story naar In Progress zetten;
- programmeren;
- commit/push;
- status bijwerken in DevOps.

Commit alleen werkende code en gebruik duidelijke commit-teksten die passen bij de user story.

Stap 2: Projectstructuur maken

Maak eerst de basisstructuur van het project.

- Models: vul deze ook alvast in – bekijk je document *modellen opzetten*
- Controllers
- Views
- Files

Stap 3: Database aanmaken met SQL

Maak de database op basis van je ERD/NORMA ontwerp.

- Maak:
- database;
- tabellen;
- primary keys;
- alternate keys;
- foreign keys;
- testdata met INSERT.

Gebruik je *database-stappenplan* als je reader/bron.

Stap 4: App.config en connection string maken

Maak de App.config aan en zet hierin de databaseverbinding. Dan commit en push je m 1 keer.

Zet daarna App.config in de .gitignore, omdat hierin jouw databaseverbinding staat.

Lees je connectionString document zodat niks fout gaat.

Stap 5: Overzichtsschermen maken voor de hoofdprocessen

Maak eerst de overzichtsschermen van de hoofdprocessen die jij moet bouwen.

Voorbeeld:

- Trainers beheren;
- Leden beheren;
- Sportpassen beheren;

Maak per overzichtsscherm (Frm) alvast je listview (je hoeft nog niks erin te laden).

Stap 6: Controllers maken + ReadAll per hoofdproces

Maak voor elk overzichtsscherm meteen een eigen controller.

Voorbeeld:

- TrainerController;
- LidController;
- SportpasController;

Maak in elke controller eerst alleen de ReadAll() method.

Controleer per hoofdproces:

werkt de SQL SELECT;

worden de gegevens in een List teruggegeven;

wordt de ListView gevuld.

Begin dus niet meteen met Create/Update/Delete. Eerst moet ReadAll + ListView laden werken.

Let op de relaties voor jouw tabellen (hoofdprocessen) – vanuit daar kan je kijken wel document je nodig hebt dus: geen relatie, 1 op veel of veel op veel.

Stap 7: Gegevens laden in de ListViews

Koppel elke ReadAll() aan het juiste overzichtsscherm en vul alvast de listview. Gebruik het document gegevens laden hiervoor.

Dag 4 – Realiseren

Stap 1: Functionaliteiten per hoofdproces afmaken

Werk per hoofdproces één voor één de volledige CRUD-functionaliteit af.

Maak dus niet eerst alle controllers tegelijk af.

dus je hebt overal de readAll method af.

Begin nu in controller 1:

1. Maak eerst de **Create()** method in de controller.
2. Maak daarna het toevoegscherm en de toevoegen-button.
3. Test of toevoegen werkt en of de ListView daarna ververs. En de data in de listview/database wordt toegevoegd.
4. Maak daarna de **Update()** method in de controller.
5. Maak het wijzigscherm en de wijzigen-button.
6. Test of wijzigen werkt en of oude gegevens goed worden geladen in de textboxes bijvoorbeeld.
7. Kijk of de data in de listview en database wordt gewijzigd.
8. Maak daarna de **Delete()** method in de controller.
9. Maak de verwijder-button in het overzicht.
10. Test of verwijderen werkt in de listview en ook kijken of het in de database is aangepast.

Pas als dit hoofdproces werkt, ga je door naar de volgende controller. En hou je dezelfde structuur aan.

Gebruik hierbij het juiste stappenplan:

- geen relatie: gewone controller + CUD buiten controller;
- 1-op-veel: controller met foreign key / objectrelatie + comboboxen;
- veel-op-veel: controller via koppeltabel + twee gekoppelde objecten/comboxen.
- Gegevens laden

Tijdens deze stap maak je automatisch ook:

- toevoegschermen;
- wijzigschermen;
- verwijderfunctionaliteit vanuit het overzicht;
- form-navigatie;
- meldingen (toevoegen gelukt/ toevoegen niet gelukt, weet u zeker dat u wilt verwijderen etc.)
- refresh van de ListView na toevoegen/wijzigen/verwijderen.

Dag 5 – realiseren

Stap 1: Werk van dag 4 afmaken

Controleer eerst of alle hoofdprocessen van dag 4 klaar zijn.

Als een hoofdproces nog niet volledig werkt, maak je dat eerst af.

Stap 2: Validatie en foutmeldingen controleren

Controleer daarna of de applicatie netjes reageert op foute invoer.

Controleer minimaal:

- lege velden;
- geen item geselecteerd;
- geen combobox-item gekozen;
- verkeerde invoer;
- verkeerde datatypes;
- databasefouten;
- verwijderen bij gekoppelde tabellen.
- Gebruik simpele duidelijke meldingen met `MessageBox.Show()`.

Let extra op relaties:

- comboboxen worden goed gevuld;
- gekozen object wordt goed opgeslagen;
- foreign keys kloppen;
- JOIN / Read laat de juiste gegevens zien;
- toevoegen/wijzigen met relaties werkt.

Bij delete met relaties moet je controleren of je eerst child-records verwijdert of netjes een foutmelding toont als verwijderen niet mag.

Stap 3: Power BI maken als dit gevraagd wordt

Alleen doen als Power BI in de opdracht staat of door de docent gevraagd wordt.

Gebruik je power bi stappenplan hiervoor.

Stap 4: Login / authenticatie verwerken

Voeg login en rollen pas toe als de hele basisapplicatie werkt.

Gebruik hiervoor je authenticatie-documenten.

Stap 5: Alles nalopen en bugs oplossen

Loop de applicatie volledig door.

Controleer:

- alle forms openen;
- alle ListViews laden data;
- Create werkt;
- Read werkt;
- Update werkt;
- Delete werkt;
- relaties werken;
- validaties werken;
- login/rollen werken;
- Power BI werkt als dit nodig is;
- er zijn geen build errors.

Los simpele bugs direct op.

Als je later bij testen/verbeteren iets nodig hebt, kun je kleine duidelijke issues gebruiken, bijvoorbeeld:

- foutmelding ontbreekt;
- ListView ververs niet;
- verkeerde knop staat nog aan;
- validatie mist bij leeg veld.

Niet expres je hele applicatie kapot maken. Alleen kleine testbare punten gebruiken als dat nodig is voor issues/verbetervoorstellen.

Stap 6: DevOps board en commits controleren

Zet user stories op de juiste status, push je code en controleer of alles klaarstaat voor de testfase.

Dag 6 – Testen

Stap 1: User stories naar Testing zetten

Als het goed is staan al je userstories op dit moment in: “In Progress”.

Pak steeds één user story die klaar is en zet deze in DevOps op Testing. Maak daarna de testcases als child van deze user story en voer de test uit.

deze volgorde aanhouden:

1. Pak User Story 1.
2. Zet deze user story naar Testing.
3. Maak de testcases als child van die user story.
4. Schrijf de testscenario's met Action en Expected result.
5. Voer de testcases uit.
6. Zet testcase op PASS of FAIL.

7. **Bij PASS:**
 - a. zet PASS in titel;
 - b. zet PASS in Discussion;
 - c. zet user story naar Done.

8. **Bij FAIL: bijvoorbeeld 2 PASS en 1 FAIL:**
 - a. Bij TC 2 zet je in Discussion wat fout ging.
 - b. Je zet FAIL in de titel van TC 2.
 - c. Je maakt een Issue als child van TC 2.
 - d. De user story gaat terug naar In Progress.
 - e. Je lost de fout op in je applicatie.
 - f. Je zet de user story opnieuw naar Testing.
 - g. Je test TC 2 opnieuw.
 - h. Als TC 2 nu goed werkt:
 - i. verander je de titel van TC 2 FAIL naar TC 2 PASS;
 - ii. zet je in Discussion bij TC 2: PASS – probleem opgelost door [korte oplossing];
 - iii. zet je in de Issue Discussion: Opgelost door [korte oplossing];
 - iv. zet je de issue op afgerond/Done als dat kan.
 - v. Als alle testcases nu PASS zijn, zet je de user story naar Done.

Dan pas naar de volgende userstory.

Reminders:

- Bij Create en Update test je meestal: correcte en complete data; incorrecte data; incomplete / missende data.
- Bij Read en Delete test je: het hoofdscenario, dus of zie je de overzicht lijst of werkt de delete etc. (hier hoeft je geen paramaters te gebruiken)

Gebruik bij de testcases de voorbeeld data uit norma.

Een foutmelding bij verkeerde invoer is juist **PASS**, als die foutmelding verwacht werd.

Bij FAIL: niet alleen zeggen “werkt niet”, maar kort opschrijven:

- wat fout gaat;
- waar het fout gaat;
- hoe je het opnieuw kunt testen;
- mogelijke oorzaak;
- mogelijke oplossing.

Zet een user story pas op **Done** als alle bijbehorende testcases PASS zijn.

(check het devops stappenplan voor onduidelijkheden)

Dag 7 – Verbeteren / opleveren

Stap 1: Applicatie kort controleren

Controleer of de applicatie klaar is om te laten zien.

Check kort:

- applicatie start zonder errors;
- belangrijkste schermen openen;
- CRUD werkt;
- relaties werken;
- testresultaten zijn zichtbaar in DevOps;
- user stories die getest zijn staan op **Done**.

Stap 2: Demo / sprint review voorbereiden

Bereid kort voor wat je gaat laten zien.

Benoem:

- welke user stories je hebt gemaakt;
- welke functionaliteiten werken;
- welke schermen je laat zien;
- welke testcases PASS/FAIL waren;
- welke issues/verbetervoorstellen er zijn.

Dit past bij de beoordeling van presenteren: je moet de functionaliteit uitleggen met een logisch en onderbouwd verhaal.

Stap 3: Onderbouwing document controleren

Controleer of je een kort Word-document hebt waarin je ontwerpkeuzes, gebruikte bronnen, privacy, security en haalbaarheid uitlegt.

Zet hierin minimaal:

- welke bronnen je hebt gebruikt;
- waarom je deze tabellen, relaties en schermen hebt gekozen;
- hoe je rekening houdt met privacy;
- hoe je veilig werkt, bijvoorbeeld met parameterized queries tegen SQL-injection;
- waarom je planning en gekozen functionaliteiten haalbaar zijn;
- welke 4 bronnen je gebruikte toen je ergens niet uitkwam.

Stap 4: Demo / sprint review uitvoeren

Demonstreer de applicatie aan docent/opdrachtgever.

Laat vooral zien:

- overzicht / Read;
- toevoegen / Create;
- wijzigen / Update;
- verwijderen / Delete;
- relaties / comboboxen;
- validatie en foutmeldingen;
- eventueel login/rollen;
- eventueel Power BI.

Beantwoord vragen kort en leg uit waarom je iets zo hebt gemaakt. Gebruik je onderbouwing document.

Stap 5: Feedback van docent/opdrachtgever noteren

Tijdens of na de demo krijg je feedback of een nieuwe wens.

Voorbeeld:

“Bij US14 – Klant toevoegen willen we dat de opslaan-knop groen wordt.”

Noteer dan:

- van wie de feedback komt;
- over welke user story het gaat;
- wat de wens precies is;
- waarom dit aangepast moet worden.

De beoordelingscriteria zeggen dat je wensen, reacties, testresultaten en meldingen moet vertalen naar realiseerbare verbetervoorstellen

Stap 6: Verbetervoorstel verwerken in DevOps

Verwerk de feedback in de bestaande user story.

Bij een nieuwe klantwens doe je dit:

1. Open de bestaande user story, bijvoorbeeld **US14 – Klant toevoegen**.
2. Voeg een nieuwe acceptance criteria toe.
3. Markeer de nieuwe acceptance criteria, bijvoorbeeld groen.
4. Zet in Discussion waarom dit is aangepast.
5. Pas de bijbehorende testcase aan.
6. Zet de user story naar de volgende sprint.
7. Zet de bijbehorende testcase ook naar de volgende sprint.
8. Neem eventuele issue mee naar de volgende sprint als die erbij hoort.

Voorbeeld Discussion:

Toegevoegd acceptance criteria 5 naar aanleiding van klantgesprek: de opslaan-knop moet groen worden.

Stap 7: Retrospective / reflectie maken

Maak of vul de retrospective in.

Zet minimaal:

- positief punt van jezelf;
- verbeterpunt van jezelf;
- positief punt van het team;
- verbeterpunt van het team;
- reactie op ontvangen feedback.